

Disciplina: Qualidade de Software e Testes

Professor: Leonardo Cardia

Diagnóstico Técnico

Gilded Rose — Modernização de Sistema Legado

Discentes:

Andre Junio Deomondes Carvalhal

Gianluca Motta Lopo

Igor Lima Carvalho

Lucca Torres Badaró Silvani

Ryan Luigi Paixão da Luz

2026

Diagnóstico Técnico — Código Legado

Análise de `legacy/gilded_rose.py` (método `update_quality`, linhas 8-36).

Code smells identificados

1. Condicionais profundamente aninhados

`legacy/gilded_rose.py:26-36` chega a 4 níveis de aninhamento (`if sell_in < 0 → if name != "Aged Brie" → if name != "Backstage..." → if quality > 0 → if name != "Sulfuras..."`). Dificulta leitura e aumenta risco de introduzir bug ao alterar uma regra — não é possível alterar uma condição sem reler todo o bloco.

2. Identificação de tipo de item por comparação de string repetida

A string "Aged Brie" aparece em 3 lugares (linhas 10, 27), "Sulfuras, Hand of Ragnaros" em 4 lugares (linhas 12, 24, 30), "Backstage passes to a TAFKAL80ETC concert" em 3 lugares (linhas 10, 17, 28). Isso é duplicação de conhecimento de domínio — se o nome de um item mudar, é preciso atualizar múltiplos pontos manualmente, com risco real de esquecer um.

3. Múltiplas responsabilidades misturadas no mesmo método

`update_quality` decide simultaneamente: direção da variação de qualidade (sobe/desce), taxa de variação, regra especial de Backstage Passes (bônus por proximidade da data) e atualização de `sell_in`. Um método que decide "o quê" e "quanto" para 5 tipos de item diferentes viola responsabilidade única.

4. Lógica de Backstage Passes embutida dentro do bloco genérico de itens "que ganham qualidade"

Linhas 17-23: a regra específica de Backstage (+1 extra com `sell_in < 11`, +1 extra adicional com `sell_in < 6`) está aninhada dentro do `else` genérico que também trata Aged Brie. Acoplamento desnecessário entre duas regras de negócio distintas.

5. Duplicação de regra de limite (`quality < 50 / quality > 0`)

A checagem `item.quality < 50` aparece 4 vezes (linhas 15, 19, 22, 35) e `item.quality > 0` aparece 3 vezes (linhas 11, 29 e implicitamente). Regra de limite de qualidade (0-50) não está centralizada — está reimplementada em cada ponto onde a `quality` é alterada.

6. Baixa testabilidade

Não há como testar a regra de "Aged Brie" isoladamente — é preciso instanciar `GildedRose` com uma lista de itens e rodar `update_quality()` completo, mesmo que o interesse seja validar apenas uma regra. Confirmado em `docs/evidencias/testes_legacy_antes.txt`: a suíte de testes existente tem cobertura real de 0% (um teste é placeholder/stub, o outro é teste de aprovação de ponta a ponta, não unitário).

7. Regra de negócio incompleta (bug confirmado)

O fixture de demonstração (`legacy/texttest_fixture.py`) já inclui um item "Conjured Mana Cake", mas `update_quality` não tem nenhuma cláusula para esse nome — ele cai no branch genérico de item comum e degrada 1/dia (2/dia após vencer), quando deveria degradar 2x mais rápido (2/dia, 4/dia após vencer). Confirmado em `docs/evidencias/execucao_legado_antes.txt` (linha "Conjured Mana Cake" perde exatamente 1 de qualidade por dia). Esse é o gap que o trabalho pede para corrigir no Passo 8.

Por que isso é um risco de manutenção e confiabilidade

- Qualquer nova regra de item (como Conjured) exige reler e potencialmente modificar um método monolítico com lógica para 4 tipos de item diferentes — alto risco de regressão em itens não relacionados à mudança.
- Sem testes unitários por tipo de item, a única forma de validar uma alteração é rodar a simulação completa e inspecionar a saída manualmente — não escala e não é repetível de forma confiável.
- Duplicação de strings de nome e de limites de quality cria múltiplos pontos de falha sincronizada: corrigir uma regra em um lugar e esquecer o espelho equivalente em outro é um erro fácil de cometer e difícil de detectar sem testes.

O que NÃO é problema (evitar diagnóstico exagerado)

- O método é relativamente curto (~30 linhas) — o problema é estrutural (acoplamento, duplicação, identificação por string), não volume de código.
- A lógica central, embora difícil de ler, está correta para os itens já existentes (confirmado por execução, `docs/evidencias/execucao_legado_antes.txt`) — a única regra de negócio ausente é a de Conjured.